Computational Fluid Dynamics
EN 530.767
HW 4

Weixuan Li

1. Objective

Consider the lid-driven cavity flow (shown in Figure 1) and develop the code that uses finite difference method to solve the problem.

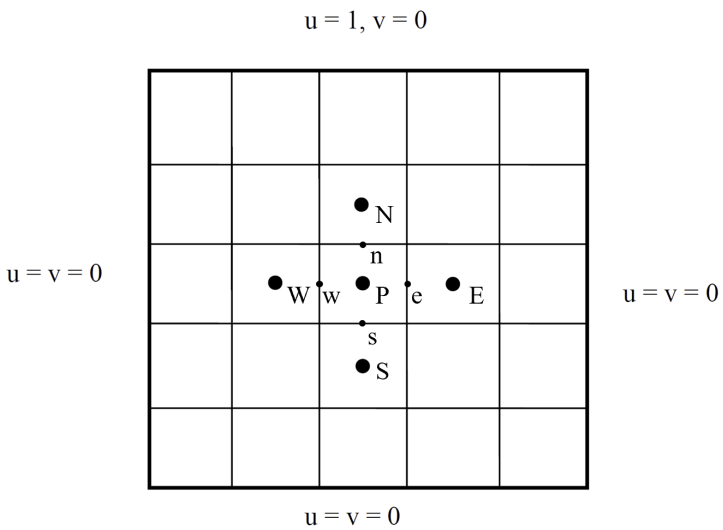


Figure 1 Configuration of the lid-driven cavity flow problem

Simulate the driven cavity flow until a steady state is reached with Re=100 and Re=1000 on uniform grids with 16, 32, 64, 128 points in each direction, and compare the results with those from previous studies.

2. Methodology

2.1 Navier-Stokes equation

The governing equation, incompressible Navier-Stokes equation, can be written in a non-dimensional form as:

$$\frac{\partial u}{\partial t} + \nabla \cdot (Uu) = -\nabla p + \frac{1}{Re}\nabla^2 u$$
$$\nabla \cdot u = 0$$

2.2 Time discretization

The time is discretized by a second-order fractional time step:

$$\frac{u^* - u^n}{\Delta t} = -(\nabla \cdot (Uu))^n + \frac{1}{Re}(\nabla^2 u)^{mix(*,n)}$$

$$\frac{u^{n+1} - u^*}{\Delta t} = -\nabla p^{n+1}$$

2.3 Algorithm procedure

1. Step 1: Calculate u^*, U^*

We calculate the medium velocity u^*, U^* by the equation:

$$\frac{u^* - u^n}{\Delta t} = -\big(\nabla \cdot (Uu)\big)^{mix(n,n-1)} + \frac{1}{Re}(\nabla^2 u)^{mix(*,n)}$$

where, the convection term is discretized by the second-order Adam-Bashforth method with staggered mesh. The numerical scheme can be written as:

$$\big(\nabla \cdot (Uu)\big)^{mix(n,n-1)}$$

$$= \frac{1}{2}\left[3 * \left(\frac{U_e \frac{u_E + u_P}{2} - U_w \frac{u_W + u_P}{2}}{\Delta x} + \frac{V_n \frac{u_N + u_P}{2} - V_s \frac{u_S + u_P}{2}}{\Delta y}\right)^n\right.$$

$$\left. - \left(\frac{U_e \frac{u_E + u_P}{2} - U_w \frac{u_W + u_P}{2}}{\Delta x} + \frac{V_n \frac{u_N + u_P}{2} - V_s \frac{u_S + u_P}{2}}{\Delta y}\right)^{n-1}\right]$$

The viscous term can be discretized by Crank–Nicolson method. The numerical scheme can be written as:

$$\frac{1}{Re}(\nabla^2 u)^{mix(*,n)}$$

$$= \frac{1}{Re}\left[\frac{1}{2}\left(\frac{u_E - 2u_P + u_W}{\Delta x^2} + \frac{u_N - 2u_P + u_S}{\Delta y^2}\right)^*\right.$$

$$\left. + \frac{1}{2}\left(\frac{u_E - 2u_P + u_W}{\Delta x^2} + \frac{u_N - 2u_P + u_S}{\Delta y^2}\right)^n\right]$$

Then, the whole equation can be discretized as

$$\left[I - \frac{\Delta t}{2Re}\nabla^2\right]u^* = -\frac{3}{2}\Delta t\big(\nabla \cdot (Uu)\big)^n + \frac{1}{2}\Delta t\big(\nabla \cdot (Uu)\big)^{n-1} + \left[I + \frac{\Delta t}{2Re}\nabla^2\right]u^n$$

$$\Longrightarrow c_W u_W + c_E u_E + c_P u_P + c_N u_N + c_S u_S = RHS$$

where

$$c_W = -\frac{\Delta t}{2Re\Delta x^2}$$
$$c_E = -\frac{\Delta t}{2Re\Delta x^2}$$
$$c_N = -\frac{\Delta t}{2Re\Delta y^2}$$
$$c_S = -\frac{\Delta t}{2Re\Delta y^2}$$
$$c_P = 1 + \frac{\Delta t}{2Re}\left(\frac{2}{\Delta x^2} + \frac{2}{\Delta y^2}\right)$$
$$RHS = -\frac{3}{2}\Delta t\big(\nabla\cdot(Uu)\big)^n + \frac{1}{2}\Delta t\big(\nabla\cdot(Uu)\big)^{n-1} + u^n + \frac{\Delta t}{2Re}\left(\frac{u_E - 2u_P + u_W}{\Delta x^2} + \frac{u_N - 2u_P + u_S}{\Delta y^2}\right)^n$$

Using line SOR to solve this matrix equation, we can obtain u^*. The details of the implementation of line SOR will be described in section 2.5. Then, calculate U^* as the average of u^*:

$$U_e{}^* = \frac{u_E^* + u_P^*}{2}, V_n{}^* = \frac{v_N^* + v_P^*}{2}$$

2. Step 2: Solve Poisson equation to obtain pressure

To make sure that the velocity satisfy the divergence free requirement, the pressure should satisfy the Poisson equation:

$$\nabla^2 p = \frac{\nabla\cdot U^*}{\Delta t} = \frac{1}{\Delta t}\left(\frac{U_e^* - U_w^*}{\Delta x} + \frac{V_n^* - V_s^*}{\Delta y}\right)$$
$$\Longrightarrow \frac{p_E - 2p_P + p_W}{\Delta x^2} + \frac{p_N - 2p_P + p_S}{\Delta y^2} = \frac{1}{\Delta t}\left(\frac{U_e^* - U_w^*}{\Delta x} + \frac{V_n^* - V_s^*}{\Delta y}\right)$$
$$\Longrightarrow c_W p_W + c_E p_E + c_P p_P + c_N p_N + c_S p_S = RHS$$

where

$$c_W = \frac{1}{\Delta x^2}$$
$$c_E = \frac{1}{\Delta x^2}$$
$$c_N = \frac{1}{\Delta y^2}$$
$$c_S = \frac{1}{\Delta y^2}$$
$$c_P = -\left(\frac{2}{\Delta x^2} + \frac{2}{\Delta y^2}\right)$$

$$RHS = \frac{1}{\Delta t}\left(\frac{U_e^* - U_w^*}{\Delta x} + \frac{V_n^* - V_s^*}{\Delta y}\right)$$

Similarly, using line SOR to solve this matrix equation, we can obtain p.

3. Step 3: Update $u^{n+1}, U^{n+1}, v^{n+1}, V^{n+1}$

$$u^{n+1} = u^* - \Delta t(\nabla p)_{cell\ center} = u^* - \Delta t\frac{p_E - p_W}{2\Delta x}$$
$$U^{n+1} = U^* - \Delta t(\nabla p)_{face\ center} = U^* - \Delta t\frac{p_E - p_P}{\Delta x}$$
$$v^{n+1} = v^* - \Delta t(\nabla p)_{cell\ center} = v^* - \Delta t\frac{p_N - p_S}{2\Delta y}$$
$$V^{n+1} = V^* - \Delta t(\nabla p)_{face\ center} = V^* - \Delta t\frac{p_N - p_P}{\Delta y}$$

4. Step 4: Calculate the residual until steady state is reached.

2.4 Boundary condition

For the velocity boundary condition, the Dirichlet boundary condition is applied on the wall, as shown in Figure 1. To achieve this, I set a layer of ghost points around the outermost layer, and let the velocity of wall be the average of the ghost point and the point right near the wall (assume this point is point P). Then

$$\frac{u_{ghost} + u_P}{2} = u_{wall}$$

For the boundary condition for pressure, I assume zero pressure gradient at wall, which is

$$\frac{\partial p}{\partial n} = 0$$

Implementing ghost points like velocity, the numerical scheme can be

$$\frac{p_P - p_{ghost}}{\Delta x} = 0$$

2.5 Implementation of Line SOR

To solve the Poisson equation for both velocity and pressure

$$\left[I - \frac{\Delta t}{2Re}\nabla^2\right]u^* = RHS;\ \nabla^2 p = RHS$$

To implement Line SOR, take the Poisson equation for velocity as an example, the discretized equation is

$$u_{i,j}^{*} - \frac{\Delta t}{2Re}\left(\frac{u_{i,j+1}^{*} - 2u_{i,j}^{*} + u_{i,j-1}^{*}}{\Delta x^2} + \frac{u_{i+1,j}^{*} - 2u_{i,j}^{*} + u_{i-1,j}^{*}}{\Delta y^2}\right) = RHS$$

$$\Rightarrow u_{i,j}^{*} - \frac{\Delta t}{2Re}\frac{u_{i,j+1}^{*} - 2u_{i,j}^{*} + u_{i,j-1}^{*}}{\Delta x^2} = RHS + \frac{\Delta t}{2Re}\frac{u_{i+1,j}^{n} - 2u_{i,j}^{n} + u_{i-1,j}^{n}}{\Delta y^2}$$

Write it in a matrix form, it can be

$$\begin{bmatrix} 1 & 0 & & & & & & \\ a & b & c & & & & & \\ & a & b & c & & & & \\ & & a & b & c & & & \\ & & & \ddots & \ddots & \ddots & & \\ & & & & a & b & c & \\ & & & & & a & b & c \\ & & & & & & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ \vdots \\ u_{n-2} \\ u_{n-1} \\ u_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ d_4 \\ \vdots \\ d_{n-2} \\ d_{n-1} \\ d_n \end{bmatrix}$$

where $b = 1 + \frac{\Delta t}{Re\Delta x^2}, a = c = -\frac{\Delta t}{2Re\Delta x^2}, d_n$ are the value on right hand side. Solve this matrix equation by TDMA and iterate until convergence, and then u^* is obtained.

3. Results and Discussions

3.1 Streamlines on Re=100 at different grid resolutions

The streamlines at the steady state with Re=100 on grids with 16, 32, 64, 128 points are shown in Figure 2. The streamlines from paper[1] Ghia et al. (Journal of Computational Physics, Vol. 48, page 387, 1982) are shown in Figure 3 to make comparison.

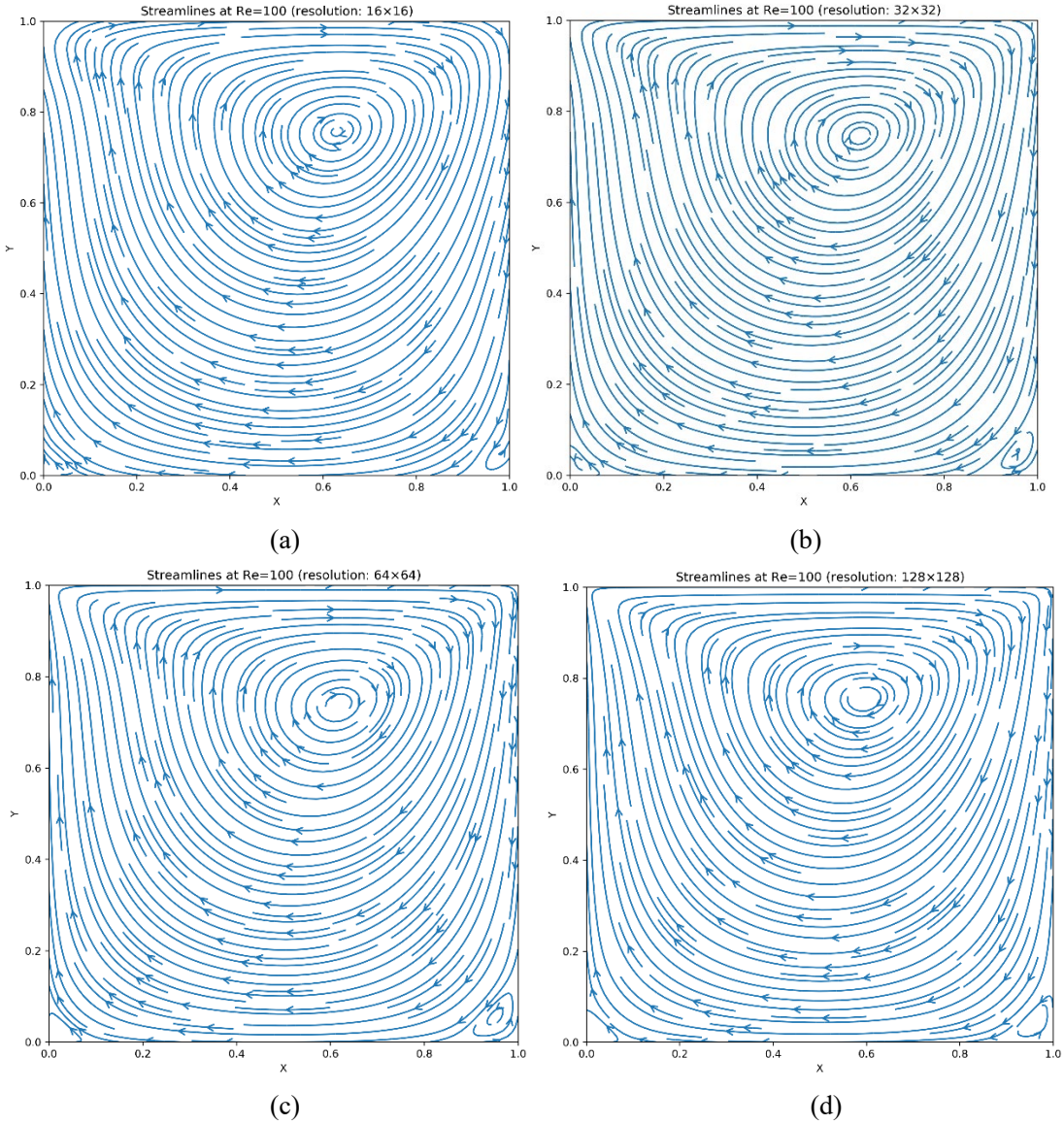


Figure 2 Streamlines with Re=100 at different grid resolutions (a) 16×16; (b) 32×32; (c) 64×64; (d) 128×128

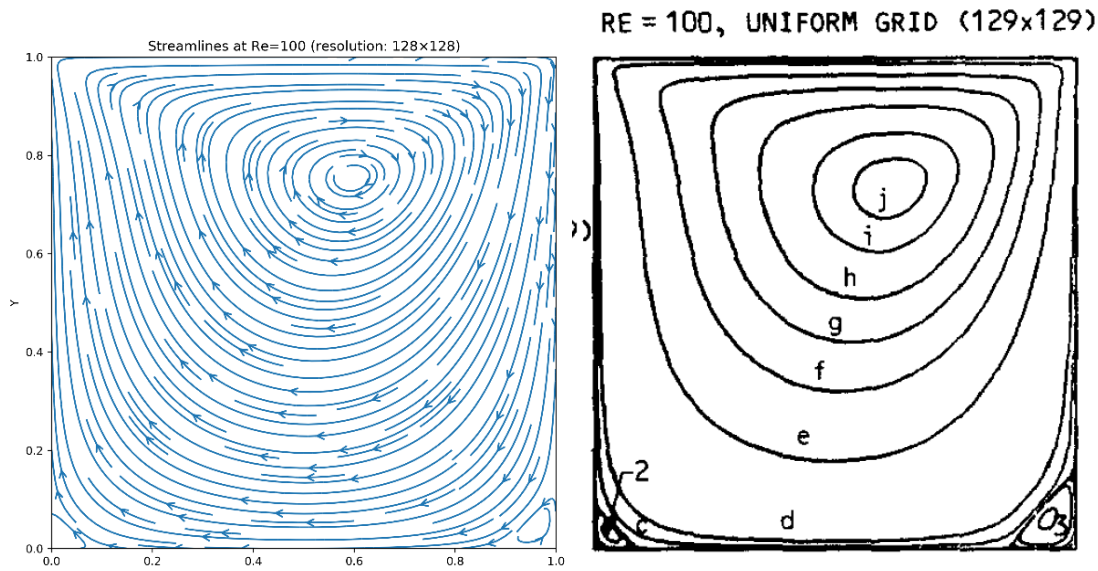


Figure 3 Comparison of streamlines at Re=100 with paper[1] Ghia et al. (*Journal of Computational Physics, Vol. 48, page 387, 1982*).

The streamlines exhibited in Figure 2, across various grid resolutions, show a high degree of correspondence with the results presented in paper[1]. There is a primary vortex that circulates clockwise, accompanied by two smaller secondary vortices at the lower corners, moving in the counter direction to the primary one. Within the primary vortex, there is a large velocity gradient that peaks at its center and diminishes gradually towards the periphery. Additionally, there sharp bends at the upper corners. A comparative analysis of streamlines across different grid resolutions reveals that finer grids capture more detailed flow structures. Notably, the secondary vortices at the lower corners are more discernible and well-structured on finer grids. While Figure 2 (a) with a 16×16 resolution only shows some lines flowing in different direction, Figure 2 (d) at a 128×128 resolution clearly exhibits secondary vortices similar to those observed in Figure 3. Furthermore, the refined grids enable a more precise capture of the sharp bends near the upper wall.

3.2 Centerline velocity at Re=100 on different grid resolutions

Figure 4 presents a comparison of centerline velocities from this study with those in paper[1]. There is a clear alignment in the trends observed between the numerical outcomes of this investigation and the reported data from paper[1]. Overall, the results demonstrate great accuracy, which is further enhanced with the refinement of the grid. On a coarser 16×16 grid, the simulation can't capture the maximal velocity very accurately, while the 128×128 grid's results align closely with the benchmark data, successfully capturing both the maximal and minimal velocity values. This trend suggests a grid convergence of accuracy. The fidelity and effectiveness of the numerical algorithm employed are underscored by these findings.

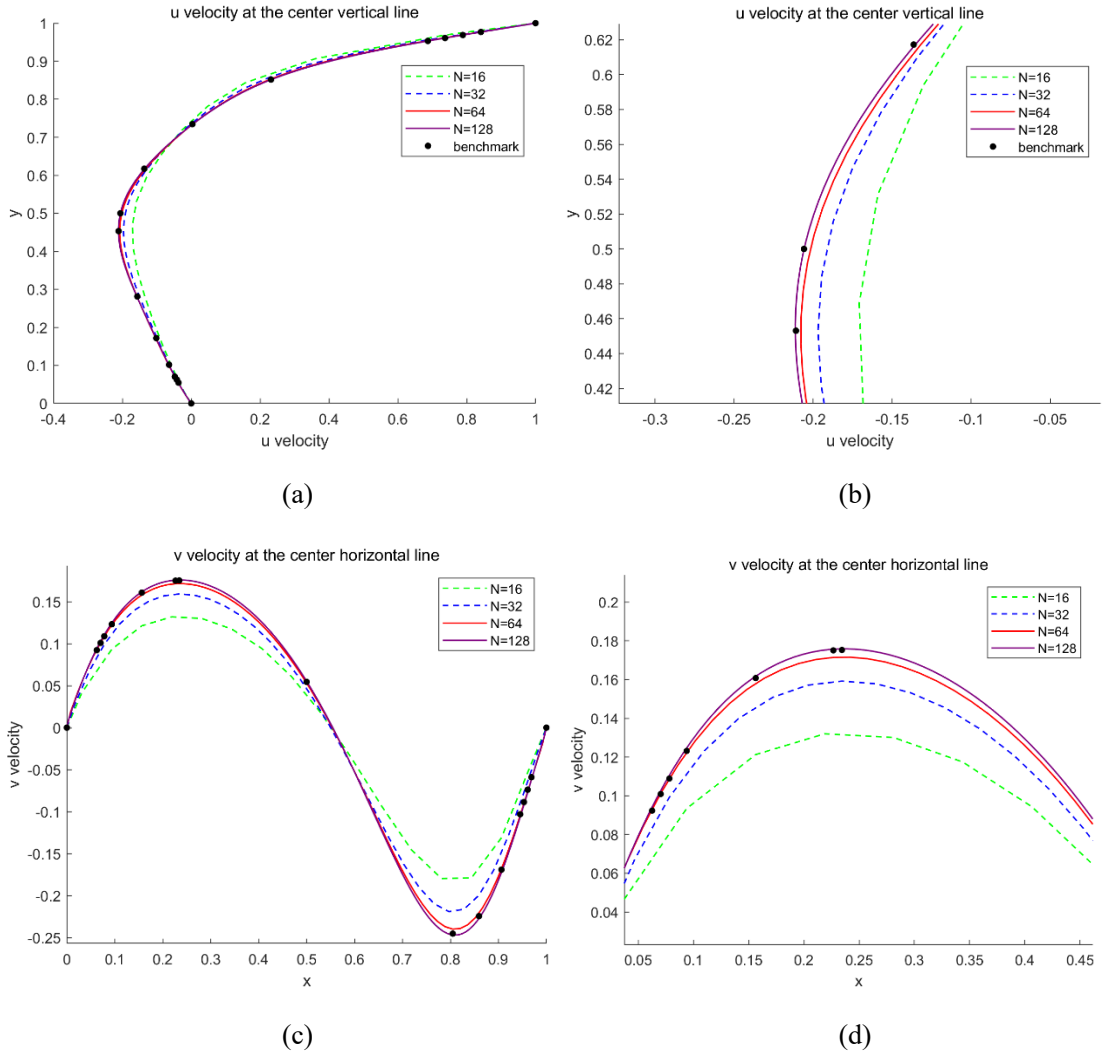


Figure 4 Comparison of centerline velocity with paper[1] when Re=100 on different grid resolutions (a) horizontal velocity at vertical center line; (b) zoom in of (a); (c) vertical velocity at horizontal center line; (d) zoom in of (c);

Reference	Grid	u_{max}	y_{max}	v_{max}	x_{max}
This study	N=128	0.2113120	0.4570	0.1758594	0.7617
Ref. [1]	N=129	0.21090	0.4531	0.17527	0.7656
Ref. [2]	N=129	0.2106	0.4531	0.1786	0.7656
Ref. [3]	N=96	0.2140424	0.4581	0.1795728	0.7630

Table 1 Comparison of the extrema of the centerline velocity with Ref. [1], Ref. [2] and Ref. [3] when Re=100

Table 1 presents a comparison of the extreme values and their corresponding positions of the velocities along the centerline between the results from this study and findings from paper[1,2,3]. The numerical data from this study closely align with results reported in paper[1,2,3], reflecting the high accuracy and effectiveness of the implemented numerical algorithm.

3.3 Streamlines and centerline velocity on Re=1000 at different grid resolutions

Like the case with Re=100, the streamlines at the steady state with Re=1000 on grids with 16, 32, 64, 128 points are shown in Figure 5. The streamlines from paper[1] at the corresponding Reynolds number are also shown in Figure 6 to make comparison.

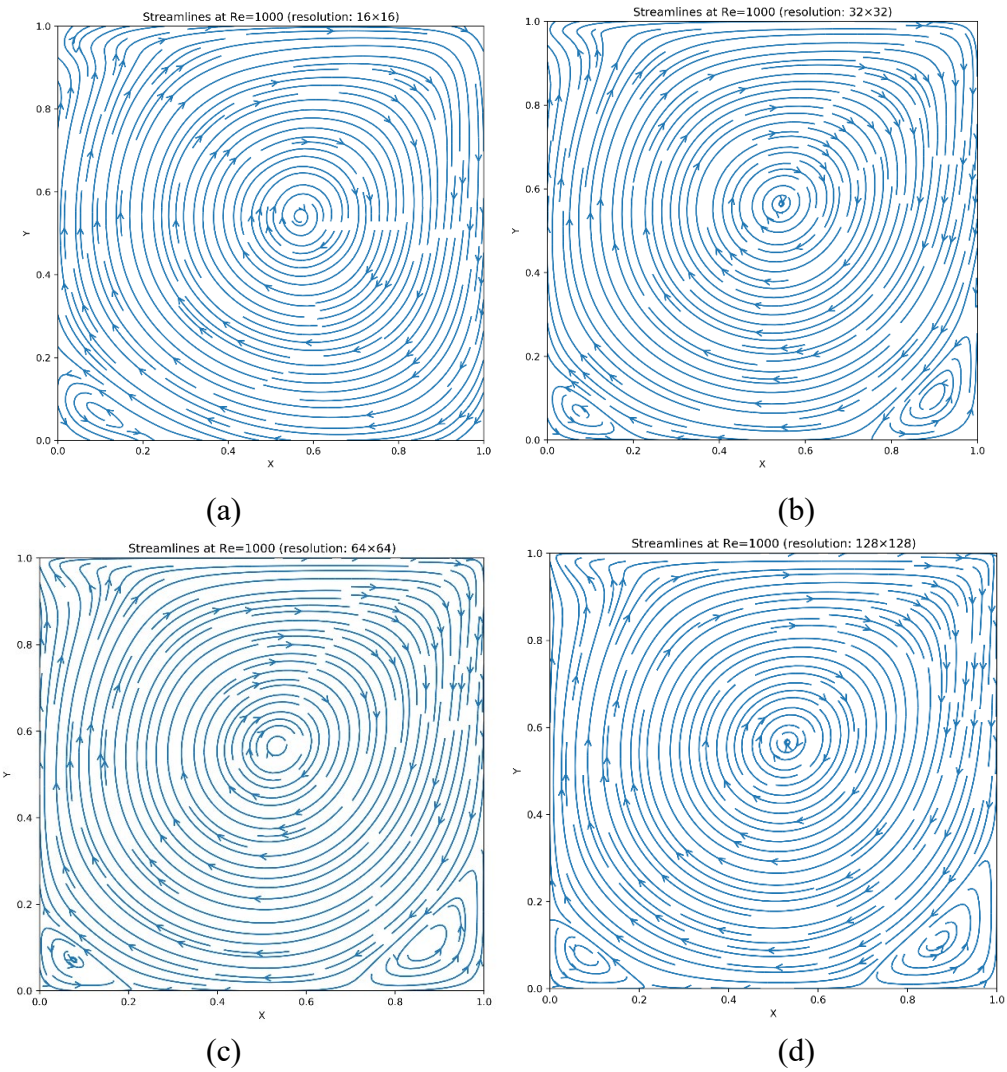


Figure 5 Streamlines with Re=1000 at different grid resolutions (a) 16×16; (b) 32×32; (c) 64×64; (d) 128×128

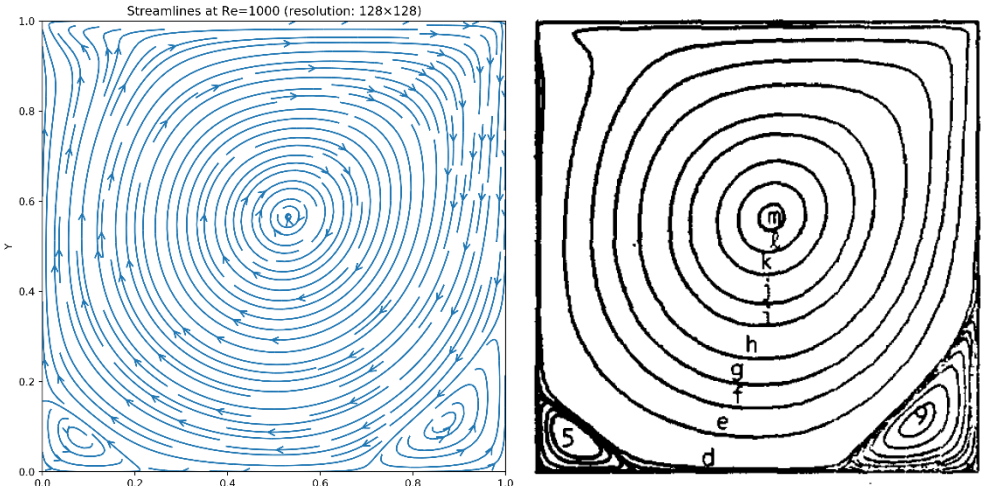
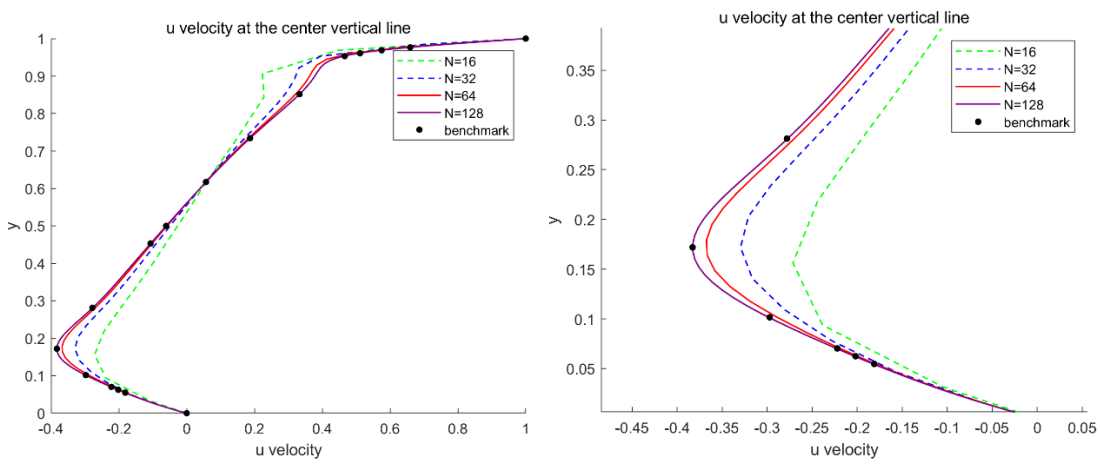
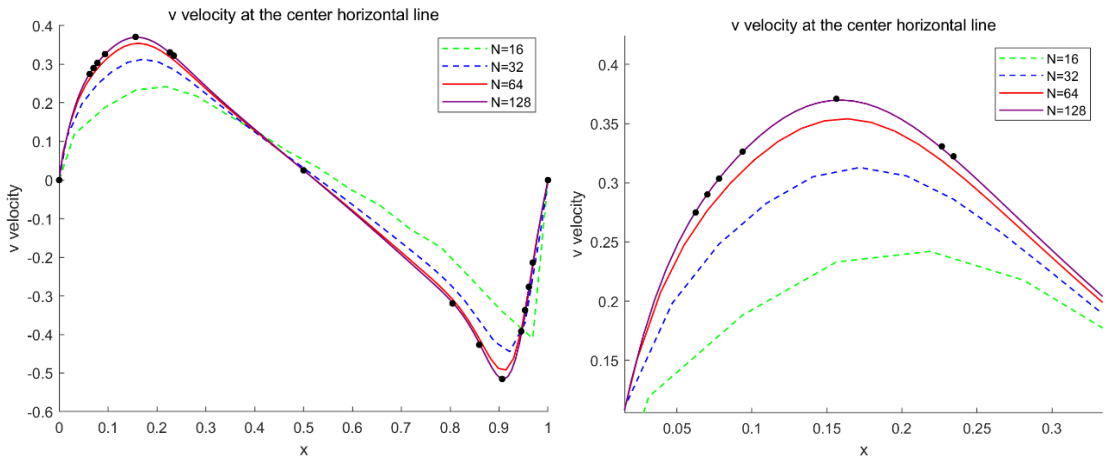


Figure 6 Streamlines with Re=1000 from paper[1] Ghia et al. (*Journal of Computational Physics, Vol. 48, page 387, 1982*).

At Re=1000, the streamlines generated by the numerical method of this study show a marked resemblance to those presented in paper[1]. Additionally, increasing the grid resolution enhances the capture of complex flow structures, particularly the secondary vortices, bringing the results closer to the benchmark (as shown in Figure 6). In comparison to the streamlines at Re=100, the secondary vortices at the bottom corners are more pronounced and larger in size, the primary vortex's center shifts downward, closer to the cavity's midpoint, and the streamlines near the upper left corner exhibit increased curvature, signifying the emergence of a third secondary vortex.



(a) (b)



(c) (d)

Figure 7 Comparison of centerline velocity with paper[1] when Re=1000 on different grid resolutions (a) horizontal velocity at vertical center line; (b) zoom in of (a); (c) vertical velocity at horizontal center line; (d) zoom in of (c);

Figure 7 displays the computed horizontal and vertical velocities at a Reynolds number of 1000. The findings of this study align closely with the outcomes reported in paper[1]. Furthermore, there is a discernible trend towards greater consistency with the benchmark as the grid resolution is refined. The outcomes at both Re=100 and Re=1000 indicate that with an adequately fine mesh (N=128 or higher), the results from this study are consistent with the published data.

4. Conclusion

This study demonstrates the successful implementation of a finite difference method to simulate lid-driven cavity flow at Reynolds numbers of 100 and 1000 across various grid resolutions. The developed code effectively compares with previous studies, showing the accuracy of the implemented numerical algorithm, as shown in Figures 3, 4 and 6, 7, where finer grids generate more accurate results. Results validate the accuracy and effectiveness of the numerical approach further through detailed comparisons of maximal velocities and vorticity with existing benchmarks, highlighted in Table 1. This study not only meets its initial objectives but also confirms the potential of the applied numerical schemes and algorithmic procedures in advancing fluid dynamics research, suggesting excellent prospects for further computational investigations into more complicated fluid behaviors.

5. References

[1] Ghia, U. K. N. G., Ghia, K. N., & Shin, C. T. (1982). High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method. Journal of computational physics, 48(3), 387-411.

[2] Bruneau, C. H., & Jouron, C. (1990). An efficient scheme for solving steady incompressible Navier-Stokes equations. Journal of Computational Physics, 89(2), 389-413.
[3] Botella, O., & Peyret, R. (1998). Benchmark spectral results on the lid-driven cavity flow. Computers & Fluids, 27(4), 421-433.

6. Appendix

MATLAB source code:

```
clear; clc;
%% Define parameters
N = 16;
dx = 1/N;
dt = 0.001;
Re = 100;
omega_u = 1;
omega_p = 1;

%% Initialization
un_m1 = zeros(N+2,N+2);
Un_m1 = zeros(N+2,N+2);
vn_m1 = zeros(N+2,N+2);
Vn_m1 = zeros(N+2,N+2);
[un_m1,Un_m1,vn_m1,Vn_m1] = BC(un_m1,Un_m1,vn_m1,Vn_m1);

un = un_m1;
Un = Un_m1;
vn = vn_m1;
Vn = Vn_m1;
p_old = zeros(N+2,N+2);
p = zeros(N+2,N+2);

%% Main loop
clc;
res = 1;
n = 0;
while res>1e-6
    t_total=tic;
    n = n+1;
    % Step 1
    NL_u =  AB2(un_m1,un,Un_m1,Un,Vn_m1,Vn,N,dx,dt);
    NL_v =  AB2(vn_m1,vn,Un_m1,Un,Vn_m1,Vn,N,dx,dt);
    dfus_u = diffusion(un,N,dt,Re,dx);
    dfus_v = diffusion(vn,N,dt,Re,dx);
```

```
tu=tic;
[u_star,U_star,iter_u] = line_sor(NL_u,dfus_u,un,dt,Re,dx,N,1,omega_u);
time_u_star = toc(tu);
tv=tic;
[v_star,V_star,iter_v] = line_sor(NL_v,dfus_v,vn,dt,Re,dx,N,2,omega_u);
time_v_star = toc(tv);
[u_star,U_star,v_star,V_star] = BC(u_star,U_star,v_star,V_star);

% Step 2
tp=tic;
[p,iter_p] = Possion(U_star,V_star,dx,dt,N,p_old,omega_p);
time_p = toc(tp);

% Step 3
[un_p1,Un_p1] = fractional_step(u_star,U_star,p,dt,dx,N,1);
[vn_p1,Vn_p1] = fractional_step(v_star,V_star,p,dt,dx,N,2);
[un_p1,Un_p1,vn_p1,Vn_p1] = BC(un_p1,Un_p1,vn_p1,Vn_p1);
umax = max(un_p1, [], 'all');
vmax = max(vn_p1, [], 'all');
cfl_u = umax*dt/dx^2;
cfl_v = vmax*dt/dx^2;
iunstable = 0;
if cfl_u > 1 || cfl_v > 1
    iunstable = 1;
end

% Step 4
tres=tic;
[max_residual,sum_residual] = residual(un_p1,un,vn_p1,vn,p,p_old);
res = sum_residual;
time_res = toc(tres);

% Update u,v
un_m1 = un;
vn_m1 = vn;
Un_m1 = Un;
Vn_m1 = Vn;

un = un_p1;
vn = vn_p1;
Un = Un_p1;
Vn = Vn_p1;
p_old = p;
time_total = toc(t_total);
```

```
    if rem(n, 1) == 0
        fprintf('n=%d, T=%f, res=%f, iter_u=%d, iter_v=%d, iter_p=%d,
iunstable=%d, cfl_u=%f, cfl_v=%f, max_residual=%f, sum_residual=%f\n', ...
         n, n*dt, res, iter_u, iter_v, iter_p, iunstable, cfl_u, cfl_v,
max_residual, sum_residual);
        fprintf('T_u=%.2f%%, T_v=%.2f%%, T_p=%.2f%%, T_res=%.2f%%\n', ...
         time_u_star/time_total * 100, time_v_star/time_total  * 100,
time_p/time_total  * 100, time_res/time_total  * 100);
    end

end

%% Save data
un_N16_Re100 = un;
vn_N16_Re100 = vn;
p_N16_Re100 = p;

%% Write tecplot file
N = 16;
[X, Y] = meshgrid(linspace(0, 1, N), linspace(0, 1, N));

fid = fopen('vel_N16_Re100.dat', 'wt');

fprintf(fid, 'TITLE = "Velocity Field"\n');
fprintf(fid, 'VARIABLES = "X" "Y" "U" "V"\n');
fprintf(fid, 'ZONE T="Zone 1", I=%d, J=%d, F=POINT\n', N, N);

for j = 2:N+1
    for i = 2:N+1
        fprintf(fid, '%f %f %f %f\n', X(i-1,j-1), Y(i-1,j-1), un(i,j),
vn(i,j));
    end
end
fclose(fid);

%% 1 Centerline velocity
% u velocity
y_com = [0, 0.0547, 0.0625, 0.0703, 0.1016, 0.1719, 0.2813, 0.4531, 0.5,
0.6172, 0.7344, 0.8516, 0.9531, 0.9609, 0.9688, 0.9766, 1];
u_com = [0, -0.03717, -0.04192, -0.04775, -0.06434, -0.10150, -0.15662, -
0.21090, -0.20581, -0.13641, 0.00332, 0.23151, 0.68717, 0.73722, 0.78871,
0.84123, 1];
color = {'g', 'b', 'r', [0.5 0 0.5], 'k'};
```

```
lineStyles = {'--', '--', '-', '-', 'none'};
markers = {'none', 'none', 'none', 'none', '.'};
figure1 = figure;
for cv = 1:5
    if cv == 1
        N=16;
        dx = 1/N;
        un = un_N16_Re100;
        vn = vn_N16_Re100;
    end
    if cv == 2
        N=32;
        dx = 1/N;
        un = un_N32_Re100;
        vn = vn_N32_Re100;
    end
    if cv == 3
        N=64;
        dx = 1/N;
        un = un_N64_Re100;
        vn = vn_N64_Re100;
    end
    if cv == 4
        N=128;
        dx = 1/N;
        un = un_N128_Re100;
        vn = vn_N128_Re100;
    end
    if cv == 5
        hold on
        plot(u_com,y_com,'linewidth',1, 'LineStyle', lineStyles{cv},
'Marker', markers{cv}, 'Color', color{cv}, 'Markersize', 15)
    else
        u_center = zeros(N+2,1);
        y_center = zeros(N+2,1);
        for i = 2:N+1
            u_center(i) = (un(i,N/2)+un(i,N/2+1))/2;
            y_center(i) = (i-1)*dx-0.5*dx;
        end
        y_center(1) = 0;
        u_center(1) = 0;
        y_center(end) = 1;
        u_center(end) = 1;
        hold on
```

```
        plot(u_center,y_center,'linewidth',1, 'LineStyle', lineStyles{cv},
'Marker', markers{cv}, 'Color', color{cv})
    end
end
ylim([0 1])
title('u velocity at the center vertical line')
xlabel('u velocity')
ylabel('y')
legend('N=16','N=32','N=64','N=128','benchmark')

% v velocity
x_com = [0, 0.0625, 0.0703, 0.0781, 0.0938, 0.1563, 0.2266, 0.2344, 0.5,
0.8047, 0.8594, 0.9063, 0.9453, 0.9531, 0.9609, 0.9688, 1];
v_com = [0, 0.09233, 0.10091, 0.10890, 0.12317, 0.16077, 0.17507, 0.17527,
0.05454, -0.24533, -0.22445, -0.16914, -0.10313, -0.08864, -0.07391, -
0.05906, 0];
figure2 = figure;
for cv = 1:5
    if cv == 1
        N=16;
        dx = 1/N;
        un = un_N16_Re100;
        vn = vn_N16_Re100;
    end
    if cv == 2
        N=32;
        dx = 1/N;
        un = un_N32_Re100;
        vn = vn_N32_Re100;
    end
    if cv == 3
        N=64;
        dx = 1/N;
        un = un_N64_Re100;
        vn = vn_N64_Re100;
    end
    if cv == 4
        N=128;
        dx = 1/N;
        un = un_N128_Re100;
        vn = vn_N128_Re100;
    end
    if cv == 5
        hold on
```

```
        plot(x_com,v_com,'linewidth',1, 'LineStyle', lineStyles{cv},
'Marker', markers{cv}, 'Color', color{cv}, 'Markersize', 15)
    else
        v_center = zeros(N+2,1);
        x_center = zeros(N+2,1);
        for i = 2:N+1
            v_center(i) = (vn(N/2,i)+vn(N/2+1,i))/2;
            x_center(i) = (i-1)*dx-0.5*dx;
        end
        x_center(1) = 0;
        v_center(1) = 0;
        x_center(end) = 1;
        v_center(end) = 0;
        hold on
        plot(x_center,v_center,'linewidth', 1,'LineStyle', lineStyles{cv},
'Marker', markers{cv},'Color', color{cv})
    end
end
title('v velocity at the center horizontal line')
xlabel('x')
ylabel('v velocity')
legend('N=16','N=32','N=64','N=128')

%% 2 Streamfunction contour
[X, Y] = meshgrid(linspace(0, 1, N), linspace(0, 1, N));
strmfn = zeros(N+2,N+2);
for j = 2:N+1
    temp1 = 0.0;
    strmfn(1, j) = 0.0;
    for i = 2:N+1
        temp1 = temp1 + 0.5 * (un(i-1, j+1) + un(i-1, j)) * dx;
        strmfn(i, j) = temp1;
    end
end
figure
contourf(X, Y, strmfn(2:N+1,2:N+1),50)
title(['Streamlines at ', sprintf('Re=%d ',Re), '(resolution: ',
sprintf('%.d',N), '\times', sprintf('%d)',N)])

%% Step 1 u_star
function [u_star,U_star,k] = line_sor(NL,dfus,un,dt,Re,dx,N,uv,omega)
    b = NL+dfus;
    R = dt/(2*Re*dx^2);
    rtot = 1;
```

```
    uk = un;
    k = 0;
    while rtot > 1e-6
        k = k+1;
        temp = uk;
        mainDiagx = (1+4*R)*ones(N,1);
        upperDiag = -R*ones(N-1, 1);
        lowerDiag = -R*ones(N-1, 1);
        triDiagMatrix = diag(mainDiagx) + diag(upperDiag, 1) +
diag(lowerDiag, -1);
        for i = 2:N+1
            B = b(i,2:N+1)+R*(temp(i-1,2:N+1)+temp(i+1,2:N+1));
            temp(i,2:N+1) = triDiagMatrix\B';
        end
        temp(2:N+1,2:N+1)=(1-
omega).*uk(2:N+1,2:N+1)+omega.*temp(2:N+1,2:N+1);
        if uv == 1
            [temp,~,~,~] = BC(temp,temp,temp,temp);
        else
            [~,~,temp,~] = BC(temp,temp,temp,temp);
        end
        uk_1 = temp;
        rtot = sum(sum(abs(uk_1-uk)));
        uk = uk_1;
    end
    u_star = uk_1;

    if uv == 1
        U_star(2:N+1,2:N+2) = (u_star(2:N+1,2:N+2)+u_star(2:N+1,1:N+1))/2;
    else
        U_star(2:N+2,2:N+1) = (u_star(2:N+2,2:N+1)+u_star(1:N+1,2:N+1))/2;
    end

end

function [un,Un,vn,Vn] = BC(un_old,Un_old,vn_old,Vn_old)
    Un_old(:,2) = 0;
    Un_old(:,end) = 0;
    Un = Un_old;

    Vn_old(2,:) = 0;
    Vn_old(end,:) = 0;
    Vn = Vn_old;
```

```
    un_old(end,:) = 2-un_old(end-1,:);
    un_old(1,:) = -un_old(2,:);
    un_old(:,1) = -un_old(:,2);
    un_old(:,end) = -un_old(:,end-1);
    un = un_old;

    vn_old(end,:) = -vn_old(end-1,:);
    vn_old(1,:) = -vn_old(2,:);
    vn_old(:,1) = -vn_old(:,2);
    vn_old(:,end) = -vn_old(:,end-1);
    vn = vn_old;

end

function NL = AB2(un_m1,un,Un_m1,Un,Vn_m1,Vn,N,dx,dt)
dy = dx;
NL = zeros(N+2,N+2);
    for i = 2:N+1
        NL(i,2:N+1) = -1/2*dt*( ...
            3*((Un(i,3:N+2).*(un(i,3:N+2)+un(i,2:N+1))/2 -
Un(i,2:N+1).*(un(i,2:N+1)+un(i,1:N))/2)/dx ...
            + (Vn(i+1,2:N+1).*(un(i+1,2:N+1)+un(i,2:N+1))/2 -
Vn(i,2:N+1).*(un(i,2:N+1)+un(i-1,2:N+1))/2)/dy ...
            ) ...
            -((Un_m1(i,3:N+2).*(un_m1(i,3:N+2)+un_m1(i,2:N+1))/2 -
Un_m1(i,2:N+1).*(un_m1(i,2:N+1)+un_m1(i,1:N))/2)/dx ...
            + (Vn_m1(i+1,2:N+1).*(un_m1(i+1,2:N+1)+un_m1(i,2:N+1))/2 -
Vn_m1(i,2:N+1).*(un_m1(i,2:N+1)+un_m1(i-1,2:N+1))/2)/dy) ...
            );
    end
end

function dfus = diffusion(un,N,dt,Re,dx)
    dy = dx;
    dfus = zeros(N+2,N+2);
    for i = 2:N+1
        dfus(i,2:N+1) = un(i,2:N+1)+dt/2/Re*((un(i,3:N+2)-
2*un(i,2:N+1)+un(i,1:N))/dx^2 ...
            + (un(i+1,2:N+1)-2*un(i,2:N+1)+un(i-1,2:N+1))/dy^2);
    end
end

%% Step 2 p
function [p,k] = Possion(U_star,V_star,dx,dt,N,p_old,omega)
```

```
    dy = dx;
    pk = p_old;
    rtot = 1;
    b = zeros(N+2,N+2);
    b(2:N+1,2:N+1) = ((U_star(2:N+1,3:N+2)-U_star(2:N+1,2:N+1))/dx +
(V_star(3:N+2,2:N+1)-V_star(2:N+1,2:N+1))/dy)/dt;
    k = 0;
    while rtot > 1e-6
        k = k+1;
        temp = pk;
        for i = 2:N+1
            temp(i,2:N+1) = -(b(i,2:N+1)*dx^2-(temp(i-
1,2:N+1)+temp(i+1,2:N+1)+temp(i,1:N)+temp(i,3:N+2)))/4;
        end
        temp(2:N+1,2:N+1)=(1-
omega).*pk(2:N+1,2:N+1)+omega.*temp(2:N+1,2:N+1);
        temp(1,2:N+1) = temp(2,2:N+1);
        temp(N+2,2:N+1) = temp(N+1,2:N+1);
        temp(2:N+1,1) = temp(2:N+1,2);
        temp(2:N+1,N+2) = temp(2:N+1,N+1);
        pk_1 = temp;
        res=zeros(N+2,N+2);
        for i = 2:N+1
            res(i,2:N+1)=pk_1(i-
1,2:N+1)+pk_1(i+1,2:N+1)+pk_1(i,1:N)+pk_1(i,3:N+2)-4*pk_1(i,2:N+1)-
b(i,2:N+1)*dx^2;
        end
        rtot = sum(sum(abs(res(2:N+1,2:N+1))));
        pk = pk_1;
    end
    p = pk_1;

end

%% step 3 un_1
function [un_1,Un_1] = fractional_step(u_star,U_star,p,dt,dx,N,uv)
    un_1 = zeros(N+2,N+2);
    Un_1 = zeros(N+2,N+2);
    pcc = zeros(N+2,N+2);
    pfc = zeros(N+2,N+2);
    if uv == 1
        pcc(2:N+1,2:N+1) = (p(2:N+1,3:N+2)-p(2:N+1,1:N))/2/dx;
        pfc(2:N+1,2:N+2) = (p(2:N+1,2:N+2)-p(2:N+1,1:N+1))/dx;
        un_1(2:N+1,2:N+1) = u_star(2:N+1,2:N+1)-dt*pcc(2:N+1,2:N+1);
```

```
        Un_1(2:N+1,2:N+2) = U_star(2:N+1,2:N+2)-dt*pfc(2:N+1,2:N+2);
    else
        pcc(2:N+1,2:N+1) = (p(3:N+2,2:N+1)-p(1:N,2:N+1))/2/dx;
        pfc(2:N+2,2:N+1) = (p(2:N+2,2:N+1)-p(1:N+1,2:N+1))/dx;
        un_1(2:N+1,2:N+1) = u_star(2:N+1,2:N+1)-dt*pcc(2:N+1,2:N+1);
        Un_1(2:N+2,2:N+1) = U_star(2:N+2,2:N+1)-dt*pfc(2:N+2,2:N+1);
    end
end

%% Step 4 Residual
function [max_residual,sum_residual] = residual(un_p1,un,vn_p1,vn,p,p_old)
    dff_u = abs(un_p1-un);
    dff_v = abs(vn_p1-vn);
    dff_p = abs(p-p_old);
    max_residual = max(dff_u + dff_v+dff_p, [], 'all');
    sum_residual = sum(sum(dff_u +dff_v+dff_p));

end
```